



PGCONF.EU 2026 · VALENCIA

Finding the invisible

Spotting query planner issues
before they appear

Andrei Lepikhov

22 October 2026

CHAPTER 1

Origins of the idea

From XXXXXXXX X. XXXXXXXX <XXXXXXXXXXXXXXXX@XXXXXX.COM>
Subject **Segmentation fault caused by Postgres - reateplan.c:6178 - prepare_sort_from_pathkeys**

2023-12-31 03:23:12.048 PST [17763] LOG: server process (PID 19014) was terminated by **signal 11: Segmentation fault**

2023-12-31 03:23:12.048 PST [17763] DETAIL: Failed process was running:

```
with recursive
pks_fks as (
    -- pk + fk referencing col
    select
        contype::text as contype,
        conname,
        array_length(conkey, 1) as ncol,
        conrelid as resorigtbl,
        col as resorigcol,
        ord
    from pg_constraint
    left join lateral unnest(conkey) with ordinality as _(col, ord)
    where contype IN ('p', 'f')
```

private chat with an AQO user (sensitive information hidden)

12 Jan 2024

Be careful using AQO – just remember, AI guesses, and these predictions can sometimes be totally wrong. The best results I achieved were when I used it manually, correcting only specific problematic queries. 1:41 ✓✓

We have turned on AQO across **5 000 servers** for all queries now 3:06

We love it. 3:06

Do you have some positive effect? 3:07 ✓✓

Yes. 3:15

Much faster across the board. 3:15

I.e. run the query 50% time without AQO then 50% w/ AQO then compare 3:18

The query plan is a great source of savings on your cloud bill.

It requires no extra hardware or watts — just a little thought and careful tuning.

The background of the slide is a light gray color. It features a pattern of thin, dark gray diagonal lines that run from the top-left towards the bottom-right. These lines are of varying lengths and are interspersed with small, dark gray dots. The overall effect is a textured, digital-like background.

CHAPTER 2

Blind spot

Is it easy to spot an optimisation problem?

One production ERP database ...

Example query

```
SELECT T1.Fld100343RRef,
       (CAST(CAST((T16.Fld100345 * T1.Fld100346_) AS NUMERIC(28,8)) /
              (T16.Fld100346_ * T1.Fld100345_) AS NUMERIC(28,8)))
FROM (SELECT T15.Fld100344RRef AS Fld100344RRef,
             T15.Fld100343RRef AS Fld100343RRef, T15.Fld100346 AS Fld100346_,
             T15.Fld100345 AS Fld100345_
FROM (SELECT T3.Fld100343RRef AS Fld100343RRef,
             T3.Fld100344RRef AS Fld100344RRef, MAX(T3._Period) AS MAXPERIOD,
             FROM _InfoRgl100342 T3
WHERE ((T3.Fld2488 = CAST(0 AS NUMERIC))) AND (T3._Period <=
'2025-12-04 12:49:02'::timestamp AND (EXISTS(SELECT 1
FROM pg_temp.tt3319 T4
LEFT OUTER JOIN _Reference101 T5
ON (T4._Q_001_F_007RRef = T5._IDRRRef) AND (T5.Fld2488 = CAST(0 AS NUMERIC)))
LEFT OUTER JOIN _Reference395 T6
ON (T4._Q_001_F_006RRef = T6._IDRRRef) AND (T6.Fld2488 = CAST(0 AS NUMERIC)))
LEFT OUTER JOIN _Reference366X1 T7
ON (T4._Q_001_F_006RRef = T7._IDRRRef) AND (T7.Fld2488 = CAST(0 AS NUMERIC)))
WHERE ((1=1 OR 1=0) AND (T4._Q_001_F_001RRef =
'216e030f\332\2618\333\021\356\222\212\342\004z\226'::bytea) AND
(T5.Fld53293RRef = '214\2150\271\351\244\310\000B<\227\003{-\356e'::bytea)
AND ((T7.Fld59861RRef <> '201Fnu\257Q\301Pp\0270\214\214\001{'::bytea)
OR (T7.Fld59861RRef = '201Fnu\257Q\301Pp\0270\214\214\001{'::bytea)))
AND ((T3.Fld100343RRef = T5.Fld53293RRef) AND
(T3.Fld100344RRef = T6.Fld101340RRef)))
UNION ALL SELECT 1
FROM pg_temp.tt3319 T8
LEFT OUTER JOIN _Reference101 T9
ON (T8._Q_001_F_007RRef = T9._IDRRRef) AND (T9.Fld2488 = CAST(0 AS NUMERIC)))
LEFT OUTER JOIN _Reference101 T10
ON (T9.Fld53293RRef = T10._IDRRRef) AND (T10.Fld2488 = CAST(0 AS NUMERIC)))
LEFT OUTER JOIN _Reference395 T11
ON (T8._Q_001_F_006RRef = T11._IDRRRef) AND (T11.Fld2488 = CAST(0 AS NUMERIC)))
LEFT OUTER JOIN _Reference366X1 T12
ON (T8._Q_001_F_006RRef = T12._IDRRRef) AND (T12.Fld2488 = CAST(0 AS NUMERIC)))
WHERE ((1=1 OR 1=0) AND (T8._Q_001_F_001RRef =
'216e030f\332\2618\333\021\356\222\212\342\004z\226'::bytea) AND
(T9.Fld53293RRef = '214\2150\271\351\244\310\000B<\227\003{-\356e'::bytea)
```

Statistics

- ~ 49500 distinct queryids
- ~ 560 queries longer than 1s.
- query text: 33 – 126 000 chars
- execution time: 1 ms – 26 s
- calls: 1 – 5 075
- blocks read: 0 – 3.5 M (≈27 GB)
- plan nodes: 1 – 195
- *never executed* nodes: 0 – 52

Closing the blind spot in query statistics

pg_stat_statements

calls	[10 000 000]
mean_exec_time	[100.0]
total_exec_time	[1 000 000 000]
rows	[10 000 000]
shared_blks_read	[50 000 000]

278 hours of execution time,
near the top of any ranking by
total time.

What we need:

- Plan quality
- Filter efficiency
- Join method choice
- Missing indexes
- work_mem impact
- ...

pg_qualstats, online_advisor, pg_stat_advisor, pganalyze ...

Assessing the instance's potential

1 Estimation error (rows estimated / rows actual)

is it worth manipulating table statistics?

2 Scan and join filter activity

do we have enough indexes?

3 SubPlans' share of the execution time

is it time to rewrite the queries?

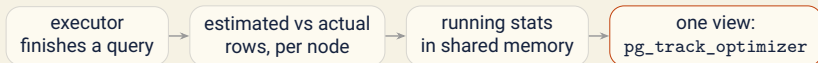
4 Spilling of intermediate results

should queries get more memory?

5 Plan stability

would a generic plan be a better idea?

Postgres extension pg_track_optimizer



11

metrics per query [ID]

0 restarts

to start tracking: shared memory via the DSM registry, no preload needed

QueryID

Modified Query ID to reduce the number of different entries

Persistent

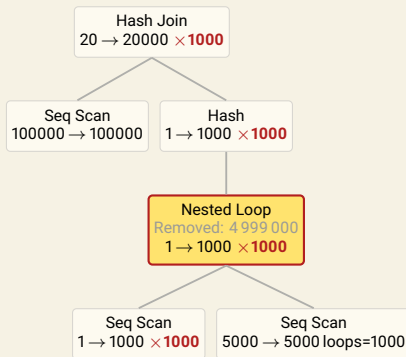
flushed to disk on backend exit, loaded back after a restart

CHAPTER 3

Domain specifics

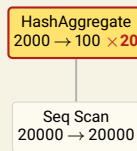
Nuance 1: Queries are different

Two joins over a filtered scan



6 nodes · 2 joins · 226 ms

GROUP BY over a scan

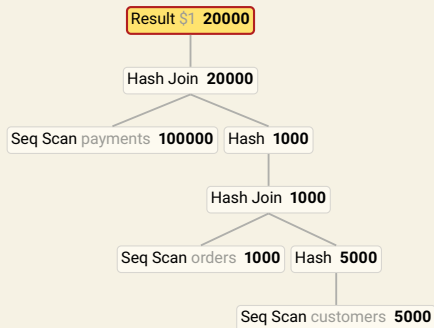


2 nodes · 0 joins · 1.4 ms

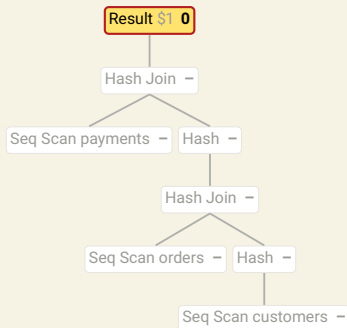
Nuance 2: Parameter might be in undefined state

```
PREPARE q(int, bool) AS  
  SELECT o.id, p.id FROM orders o  
    JOIN customers c ON c.id = o.cust JOIN payments p ON p.cust = c.id  
  WHERE $1;
```

EXECUTE q(true)



EXECUTE q(false)



Nuance 3: statistics' stochastic nature

```
-- 40 times, the data never change  
ANALYZE t;  
-- actual: 10000 rows every time  
EXPLAIN SELECT * FROM t WHERE x = 77;
```

77 made it into the MCV list

≈ 10 000 rows

Bitmap Heap Scan

21 runs of 40

77 missed the MCV list

≈ 60 rows, **off by ×170**

Index Scan

19 runs of 40

Two design decisions in pg_track_optimizer

Dimensionless metrics:

per plan node, units cancel out:

- $e = \left| \ln \frac{\text{actual rows}}{\text{estimated rows}} \right|$
- $r = \frac{\text{node time}}{\text{total time}}$
- $f = \frac{\text{filtered rows}}{\text{actual rows}}$
- $b = \frac{\text{local blocks}}{\text{total blocks}}$

Custom RStats type:

a distribution, not a number:

(count, mean, min, max, stddev)

- Updated in place on every execution (Welford): the plan is unstable, not just wrong.
- The same code path for each parameter.
- Aggregate on statistic column.

CHAPTER 4

How it works

How does it work in practice?

Test bed: **the Join Order Benchmark**

Leis et al., *How Good Are Query Optimizers, Really?*, PVLDB 2015

113

queries over real IMDb
data

17

tables joined in the
deepest query

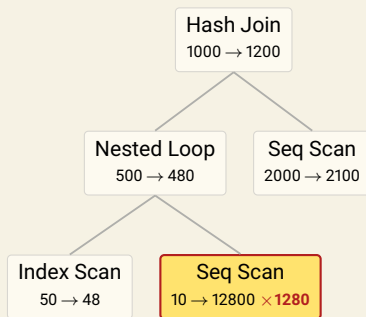
Scans

many of them, including
parameterised ones

Room

plenty of it, for the planner
to go wrong

Cardinality estimation error



per-node error:

$$e = \left| \ln \frac{\text{actual}}{\text{estimated}} \right|$$

How to average across plan nodes?

How the estimation error criteria work

TOP-10 by average error V/S TOP-10 by time-weighted error

```
SELECT avg.* FROM
  (SELECT queryid, query, avg_avg, twa_avg FROM pg_track_optimizer
   ORDER BY avg_avg DESC LIMIT 10) AS avg
JOIN
  (SELECT queryid FROM pg_track_optimizer
   ORDER BY twa_avg DESC LIMIT 10) AS twa
USING (queryid);
```

queryid	query	avg_avg	twa_avg
-6836119621424365027	/* 28a.sql */	3.91	2.16
3631768377448547020	/* 29c.sql */	3.77	2.51
-8544699698054799639	/* 22d.sql */	3.48	2.09
4682104415514032107	/* 7c.sql */	3.48	2.18

(4 rows)

A look inside 28a.sql

12.	0.000	0.000	↓ 15,866.0		1	→ Nested Loop (cost=673,574.16..713,608.92 rows=1 width=65) (actual rows=15,866.00 loops=1)
13.	0.000	0.000	↓ 2,641,497.5		1	→ Parallel Hash Join (cost=673,573.73..713,604.75 rows=2 width=69) (actual rows=5,282,995.00 loops=1) Hash Cond: (mk.movie_id = t.id)
14.	0.000	0.000	↓ 4.0		1	→ Parallel Seq Scan on movie_keyword mk (cost=0.00..35,789.82 rows=1,130,982 width=8) (actual rows=4,523,930.00 loops=1)
15.	0.000	0.000	↓ 47,805.0		1	→ Parallel Hash (cost=673,573.72..673,573.72 rows=1 width=61) (actual rows=47,805.00 loops=1) Buckets: 65,536 (originally 1024) Batches: 1 (originally 1) Memory Usage: 5,368kB
16.	0.000	0.000	↓ 47,805.0		1	→ Nested Loop (cost=611,718.06..673,573.72 rows=1 width=61) (actual rows=47,805.00 loops=1) Join Filter: (t.id = mk.movie_id)
17.	0.000	0.000	↓ 18,042.0		1	→ Nested Loop (cost=611,717.63..673,566.96 rows=3 width=40) (actual rows=54,126.00 loops=1)
18.	0.000	0.000	↓ 7,874.1		1	→ Parallel Hash Join (cost=611,717.20..673,551.17 rows=7 width=28) (actual rows=55,119.00 loops=1) Hash Cond: (pi.person_id = ci.person_id)
19.	0.000	0.000	↓ 94.7		1	→ Parallel Hash Join (cost=1.84..61,811.22 rows=6,553 width=4) (actual rows=620,526.00 loops=1) Hash Cond: (pi.info_type_id = it3.id)
20.	0.000	0.000	↓ 4.0		1	→ Parallel Seq Scan on person_info pi (cost=0.00..59,792.74 rows=740,474 width=8) (actual rows=2,963,664.00 loops=1)
21.	0.000	0.000	↑ 1.0		1	→ Parallel Hash (cost=1.83..1.83 rows=1 width=4) (actual rows=1.00 loops=1) Buckets: 1,024 Batches: 1 Memory Usage: 40kB
22.	0.000	0.000	↑ 1.0	- 112	1	→ Parallel Seq Scan on info_type it3 (cost=0.00..1.83 rows=1 width=4) (actual rows=1.00 loops=1) Filter: ((info)::text = 'trivia'::text) Rows Removed by Filter: 112

```

-> Parallel Hash Join (rows=6553) (actual rows=620526)
    Hash Cond: (pi.info_type_id = it3.id)
-> Parallel Seq Scan on person_info pi (rows=740474) (actual rows=2963664)
-> Parallel Hash (rows=1) (actual rows=1.00)
    -> Parallel Seq Scan on info_type it3 (rows=1) (actual rows=1)
        Filter: ((info)::text = 'trivia'::text)
        Rows Removed by Filter: 112

```

⇒ The x4 scan error grows throughout the join tree.

Filter factors: scans and joins

Scans

$$f_{scan} = \max_{scans} \left(\frac{\text{removed by filter}}{\text{actual rows}} \times \frac{\text{node time}}{\text{total time}} \right)$$

Joins

$$f_{join} = \max_{joins} \left(\frac{\text{removed by join filter}}{\text{actual rows}} \times \frac{\text{node time}}{\text{total time}} \right)$$

Hash Join (actual time=234.671 rows=289 loops=1)

Hash Cond: (o.customer_id = c.id)

Join Filter: (o.amount > c.credit_limit)

Rows Removed by Join Filter: 47542

-> Seq Scan on orders o (actual time=98.341 rows=47831 loops=1)

Filter: (status = 'completed')

Rows Removed by Filter: 952169

-> Hash (actual time=42.110 rows=100000 loops=1)

-> Seq Scan on customers c (actual time=21.543 rows=100000)

How the filter criteria work

```
SELECT queryid, query, avg_avg, lf_avg
FROM pg_track_optimizer
ORDER BY lf_avg DESC LIMIT 5;
```

queryid	query	avg_avg	lf_avg
7987931480126446697	/* 15b.sql */	1.60	1459.35
-4244378222084090458	/* 23a.sql */	1.75	1424.58
-6679854959453081533	/* 15a.sql */	1.95	1406.36
-1139306376886509831	/* 23b.sql */	1.48	1403.23
-4904731422305343909	/* 15c.sql */	2.69	1342.81

[illegible]

How the filter criteria work

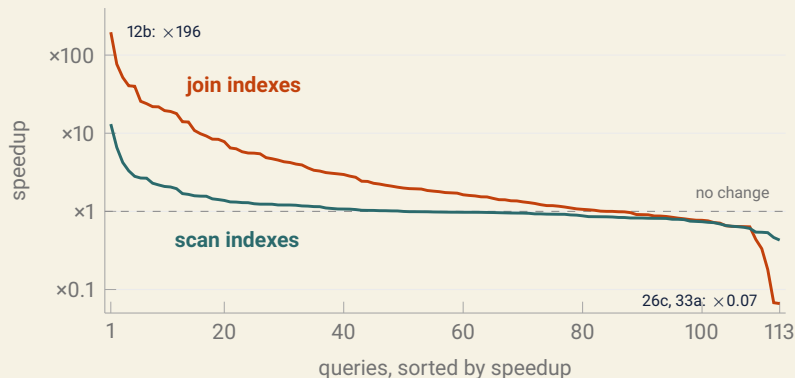
```
SELECT queryid, query, avg_avg, lf_avg
FROM pg_track_optimizer
ORDER BY lf_avg DESC LIMIT 5;
```

queryid	query	avg_avg	lf_avg
7987931480126446697	/* 15b.sql */	1.60	1459.35
-4244378222084090458	/* 23a.sql */	1.75	1424.58
-6679854959453081533	/* 15a.sql */	1.95	1406.36
-1139306376856509831	/* 23b.sql */	1.48	1403.23
-4904731422305343909	/* 15c.sql */	2.69	1342.81

[illegible]

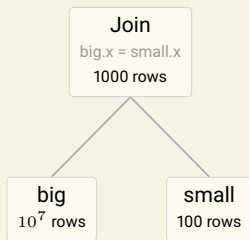
```
-> Parallel Hash Join (actual rows=354.20 loops=5)
    Hash Cond: (mi.info_type_id = itl.id)
    -> Parallel Seq Scan on movie_info mi (actual rows=354.20 loops=5)
        Filter: ((note ~ '%internet%':text) AND (info ~ 'USA:% 200%':text))
        Rows Removed by Filter: 2966790
    -> Parallel Hash (actual rows=0.20 loops=5)
```

Effect of the indexes found



JOB, 113 queries: plain time / time with the new indexes, best to worst

Join filtering EXPLAIN can't see



```
SELECT * FROM big
JOIN small USING (x);
```

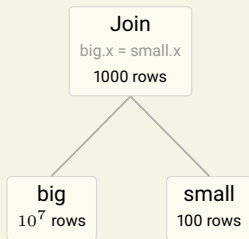
Hash Join

```
Hash Join (actual rows=1000.00 loops=1)
  Hash Cond: (big.x = small.x)
    -> Seq Scan on big (actual rows=10000000.00 loops=1)
    -> Hash (actual rows=100.00 loops=1)
        -> Seq Scan on small (actual rows=100.00 loops=1)
  Execution Time: 332.203 ms
```

Nested Loop

```
Nested Loop (actual rows=1000.00 loops=1)
  Join Filter: (big.x = small.x)
  Rows Removed by Join Filter: 999999000
    -> Seq Scan on big (actual rows=10000000.00 loops=1)
    -> Materialize (actual rows=100.00 loops=10000000)
        -> Seq Scan on small (actual rows=100.00 loops=1)
  Execution Time: 20799.413 ms
```

Join filtering EXPLAIN can't see



```
SELECT * FROM big  
JOIN small USING (x);
```

```
CREATE INDEX idx ON big (x);
```

Hash Join

```
Hash Join (actual rows=1000.00 loops=1)  
  Hash Cond: (big.x = small.x)  
    -> Seq Scan on big (actual rows=10000000.00 loops=1)  
    -> Hash (actual rows=100.00 loops=1)  
          -> Seq Scan on small (actual rows=100.00 loops=1)  
Execution Time: 332.203 ms
```

Nested Loop

```
Nested Loop (actual rows=1000.00 loops=1)  
  -> Seq Scan on small (actual rows=100.00 loops=1)  
  -> Index Scan using idx on big  
        (actual rows=10.00 loops=100)  
        Index Cond: (x = small.x)  
        Index Searches: 100  
Execution Time: 0.187 ms
```

Spilling of intermediate results

Who spills:

- Hash Join
- Hash Aggregate
- Window Aggregate
- Sort
- Materialize
- CTE /Recursive CTE

Who doesn't spill:

- INTERSECT/EXCEPT (Hashed)
- Memoize
- Hashed SubPlan
- BitmapScan

How the spilling criterion works

Rank queries by how much they spill, then see if memory actually matters

```
SELECT queryid, LEFT(query,13) AS query,  
       ROUND((temp_avg*8192/1024/1024)  
             ::numeric,0) AS spill_mb  
FROM track_data  
ORDER BY CASE WHEN blks_avg = 0 THEN 0  
              ELSE temp_avg/blks_avg END DESC  
LIMIT 5;
```

queryid	query	spill_mb
-4733402157366146115	/* 8c.sql */	598
-3437698612165670854	/* 8d.sql */	381
4682104415514032107	/* 7c.sql */	2678
1798119524484989875	/* 10c.sql */	179
-3013169471333993680	/* 16c.sql */	91

(5 rows)

Same query, only work_mem
changes

```
SET work_mem = '1MB';  
Execution Time: 1841 ms
```

```
SET work_mem = '4MB';  
Execution Time: 1405 ms
```

```
SET work_mem = '64MB';  
Execution Time: 950 ms
```

What a bigger `work_mem` changes

- **Hash Join** → **Nested Loop + Memoize**
- Incremental Sort + Merge Append → Sort + Append
- Merge Append + Sort → Sort + Append
- **Group Aggregate + Sort** → **Hash Aggregate**
- Index Scan → Sort + Seq Scan
- Index Scan → Incremental Sort + Index Scan
- **Hash Join** → **Merge Join**
- → Materialize

The background of the slide is a light beige color. It features a complex pattern of thin, dark grey diagonal lines and small dots. These elements are arranged in a way that creates a sense of depth and movement, particularly on the right side of the slide where the pattern is more dense and layered.

CHAPTER 5

Outcomes

(the shortest one)

The useful bits

1 Track accumulated average error

On tuned system growth or spike in average error means loosing efficiency

2 Watch SCAN and JOIN Filter factors

The most critical metrics that expose missed indexes

3 Combine these metrics with `pg_stat_statements`

It might highlight queries that deserve GENERIC/CUSTOM plan changeover

Questions?

Andrei Lepikhov

pgEdge

lepihov@gmail.com

github.com/danolivo

pgedge.com



[pg_track_optimizer](#) on GitHub